

codeHive

Merge Sort

DIVIDE. CONQUER. UNIFIED.

DSA Mastery Series: Vol 13

codeHive

1. Divide & Conquer

Merge Sort is a powerful, recursive sorting algorithm that follows the **divide-and-conquer** paradigm. Instead of sorting in place, it breaks the problem into tiny pieces and reassembles them in perfect order.

DIVIDE

Split the array in half until each sub-array has only 1 element.

CONQUER

Merge the small pieces back together, comparing values to ensure order.

Why Merge Sort?

Unlike Quicksort, which can perform poorly on sorted data, Merge Sort is incredibly stable and predictable. It splits and merges with the same efficiency regardless of the input's initial order.

2. Manual Architecture

Trace: [12, 8, 9, 3, 11, 5, 4]

Phase A: The Split

```
[ 12, 8, 9 ] | [ 3, 11, 5, 4 ]
```

```
[ 12 ] [ 8, 9 ] | [ 3, 11 ] [ 5, 4 ]
```

```
[ 12 ] [ 8 ] [ 9 ] | [ 3 ] [ 11 ] [ 5 ] [ 4 ]
```

Phase B: The Unified Merge

Compare neighbors and merge:

```
[ 12 ] [ 8, 9 ] | [ 3, 11 ] [ 4, 5 ]
```

```
[ 8, 9, 12 ] | [ 3, 4, 5, 11 ]
```

Final Merge with Pointer Comparison:

```
3 < 8 → [ 3, ... ]
```

```
4 < 8 → [ 3, 4, ... ]
```

```
5 < 8 → [ 3, 4, 5, ... ]
```

```
8 < 11 → [ 3, 4, 5, 8, ... ]
```

```
Final: [ 3, 4, 5, 8, 9, 11, 12 ]
```

3. Recursive Implementation

This is the standard approach, using recursion to handle the splitting and a helper function to merge.

```
def mergeSort(arr):
    if len(arr) ≤ 1:
        return arr

    mid = len(arr) // 2
    leftHalf = arr[:mid]
    rightHalf = arr[mid:]

    # Recursive calls
    sortedLeft = mergeSort(leftHalf)
    sortedRight = mergeSort(rightHalf)

    return merge(sortedLeft, sortedRight)

def merge(left, right):
    result = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])
    return result
```

4. Non-Recursive (Iterative)

While recursion is intuitive, Merge Sort can be implemented using a while loop and a growing step size to avoid function call overhead.

```
def mergeSortIterative(arr):
    step = 1
    length = len(arr)

    while step < length:
        for i in range(0, length, 2 * step):
            left = arr[i : i + step]
            right = arr[i + step : i + 2 * step]

            merged = merge(left, right)

            for j, val in enumerate(merged):
                arr[i + j] = val

        step *= 2
    return arr
```

5. Performance & Constraints

TIME COMPLEXITY

$O(n \log n)$

Consistently efficient across Best, Worst, and Average cases.

Comparison Table

Algorithm	Time Complexity	Stability
Bubble Sort	$O(n^2)$	Stable
Quicksort	$O(n \log n)$ average	Unstable
Merge Sort	$O(n \log n)$	Stable

6. Expert Insight

Merge Sort is the primary choice for sorting **linked lists** because it doesn't require random access memory (unlike Quicksort). It is also highly **parallelizable**, meaning it can be multi-threaded to sort massive datasets across multiple CPU cores simultaneously.

© 2026 codeHive Academy • Mastering Divide and Conquer